LAB 4: XÂY DỰNG AI AGENT ĐẦU TIÊN VỚI LANGGRAPH

Thời lượng: 240 Phút (4 Giờ)

Bối cảnh:

Bạn được giao nhiệm vụ xây dựng một "Trợ lý Du lịch Thông minh" cho startup TravelBuddy.Agent này giúp người dùng lên kế hoạch chuyến đi bằng cách tự động tra cứu chuyến bay, kiểm tra ngân sách, và tìm kiếm khách sạn phù hợp. Điểm đặc biệt: Agent phải biết KẾT HỢP thông tin từ nhiều nguồn để đưa ra gợi ý tối ưu — không chỉ trả lời từng câu rời rạc.

Ví dụ: Khi user nói "Tôi muốn đi Đà Nẵng cuối tuần này, budget 5 triệu", Agent phải tự tra vé máy bay, tính chi phí còn lại, rồi tìm khách sạn vừa túi tiền — tất cả trong một cuộc hội thoại.

==========

PHẦN 0: SETUP MÔI TRƯỜNG (30 Phút)

==========

Mục tiêu: Đảm bảo gọi được API thành công trước khi vào bài.

1. Khởi tạo dự án & Cài đặt thư viện

```
mkdir lab4_agent && cd lab4_agent
python -m venv venv
source venv/bin/activate          # Mac/Linux
# venv\Scripts\activate           # Windows
pip install langchain langchain-openai langgraph python-dotenv
```

2. Cấu hình API Key — tạo file .env:

```
OPENAI_API_KEY=sk-proj-xxxxxxxxxxxxxxxxxxxxxxxx
```

3. Sanity Check — tạo file test_api.py:

```
import os
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI
```

```
load_dotenv()
llm = ChatOpenAI(model="gpt-4o-mini")
print(llm.invoke("Xin chào?").content)
```

Chạy: python test_api.py — nếu in ra câu trả lời, có thể bắt đầu bài tập.

==========

PHẦN 1: THIẾT KẾ SYSTEM PROMPT (45 Phút)

==========

Mục tiêu: Định hình "Não bộ" của Agent — cách nó suy nghĩ và hành xử.

Nhiệm vụ: Tạo file system_prompt.txt theo cấu trúc XML gồm các phần sau:

```
<persona>
Bạn là trợ lý du lịch của TravelBuddy — thân thiện, am hiểu du lịch Việt
Nam, và luôn tư vấn dựa trên ngân sách thực tế của khách hàng. Bạn nói
chuyện tự nhiên như một người bạn đi du lịch nhiều, không robot.
</persona>

<rules>
1. Trả lời bằng tiếng Việt.
2.
</rules>

<tools_instruction>
Bạn có 3 công cụ:
- search_flights:
- search_hotels:
- calculate_budget:
</tools_instruction>

<response_format>
Khi tư vấn chuyến đi, trình bày theo cấu trúc:
Chuyến bay: ...
Khách sạn: ...
Tổng chi phí ước tính: ...
```

```
Gợi ý thêm: ...
</response_format>

<constraints>
- Từ chối mọi yêu cầu không liên quan đến du lịch/đặt phòng/đặt vé
 (VD: viết code, làm bài tập, tư vấn tài chính, chính trị).
-
</constraints>
```

Bài tập: Dựa trên mẫu trên, hãy tùy chỉnh và bổ sung thêm rules

hoặc constraints mà bạn cho là cần thiết. Giải thích lý do trong comment.

==========

PHẦN 2: LẬP TRÌNH CUSTOM TOOLS (45 Phút)

==========

Mục tiêu: Thiết kế "Tay chân" cho Agent — 3 tools với mock data có mối liên hệ.

Nhiệm vụ: Tạo file tools.py

```
--- BẮT ĐẦU CODE tools.py ---

from langchain_core.tools import tool

# ==========================================================
# MOCK DATA — Dữ liệu giả lập hệ thống du lịch
# Lưu ý: Giá cả có logic (VD: cuối tuần đắt hơn, hạng cao hơn đắt hơn)
# Sinh viên cần đọc hiểu data để debug test cases.
# ==========================================================

FLIGHTS_DB = {
    ("Hà Nội", "Đà Nẵng"): [
        {"airline": "Vietnam Airlines", "departure": "06:00", "arrival":
"07:20", "price": 1_450_000, "class": "economy"},
        {"airline": "Vietnam Airlines", "departure": "14:00", "arrival":
"15:20", "price": 2_800_000, "class": "business"},
```

```
        {"airline": "VietJet Air",      "departure": "08:30", "arrival":
"09:50", "price": 890_000,   "class": "economy"},
        {"airline": "Bamboo Airways",   "departure": "11:00", "arrival":
"12:20", "price": 1_200_000, "class": "economy"},
    ],
    ("Hà Nội", "Phú Quốc"): [
        {"airline": "Vietnam Airlines", "departure": "07:00", "arrival":
"09:15", "price": 2_100_000, "class": "economy"},
        {"airline": "VietJet Air",      "departure": "10:00", "arrival":
"12:15", "price": 1_350_000, "class": "economy"},
        {"airline": "VietJet Air",      "departure": "16:00", "arrival":
"18:15", "price": 1_100_000, "class": "economy"},
    ],
    ("Hà Nội", "Hồ Chí Minh"): [
        {"airline": "Vietnam Airlines", "departure": "06:00", "arrival":
"08:10", "price": 1_600_000, "class": "economy"},
        {"airline": "VietJet Air",      "departure": "07:30", "arrival":
"09:40", "price": 950_000,   "class": "economy"},
        {"airline": "Bamboo Airways",   "departure": "12:00", "arrival":
"14:10", "price": 1_300_000, "class": "economy"},
        {"airline": "Vietnam Airlines", "departure": "18:00", "arrival":
"20:10", "price": 3_200_000, "class": "business"},
    ],
    ("Hồ Chí Minh", "Đà Nẵng"): [
        {"airline": "Vietnam Airlines", "departure": "09:00", "arrival":
"10:20", "price": 1_300_000, "class": "economy"},
        {"airline": "VietJet Air",      "departure": "13:00", "arrival":
"14:20", "price": 780_000,   "class": "economy"},
    ],
    ("Hồ Chí Minh", "Phú Quốc"): [
        {"airline": "Vietnam Airlines", "departure": "08:00", "arrival":
"09:00", "price": 1_100_000, "class": "economy"},
        {"airline": "VietJet Air",      "departure": "15:00", "arrival":
"16:00", "price": 650_000,   "class": "economy"},
    ],
}

HOTELS_DB = {
    "Đà Nẵng": [
```

```
        {"name": "Mường Thanh Luxury",    "stars": 5, "price_per_night": 1_800_000, "area": "Mỹ Khê",      "rating": 4.5},
        {"name": "Sala Danang Beach",     "stars": 4, "price_per_night": 1_200_000, "area": "Mỹ Khê",      "rating": 4.3},
        {"name": "Fivitel Danang",        "stars": 3, "price_per_night": 650_000,   "area": "Sơn Trà",     "rating": 4.1},
        {"name": "Memory Hostel",         "stars": 2, "price_per_night": 250_000,   "area": "Hải Châu",    "rating": 4.6},
        {"name": "Christina's Homestay",  "stars": 2, "price_per_night": 350_000,   "area": "An Thượng",   "rating": 4.7},
    ],
    "Phú Quốc": [
        {"name": "Vinpearl Resort",       "stars": 5, "price_per_night": 3_500_000, "area": "Bãi Dài",     "rating": 4.4},
        {"name": "Sol by Meliá",          "stars": 4, "price_per_night": 1_500_000, "area": "Bãi Trường",  "rating": 4.2},
        {"name": "Lahana Resort",         "stars": 3, "price_per_night": 800_000,   "area": "Dương Đông",  "rating": 4.0},
        {"name": "9Station Hostel",       "stars": 2, "price_per_night": 200_000,   "area": "Dương Đông",  "rating": 4.5},
    ],
    "Hồ Chí Minh": [
        {"name": "Rex Hotel",             "stars": 5, "price_per_night": 2_800_000, "area": "Quận 1",      "rating": 4.3},
        {"name": "Liberty Central",       "stars": 4, "price_per_night": 1_400_000, "area": "Quận 1",      "rating": 4.1},
        {"name": "Cochin Zen Hotel",      "stars": 3, "price_per_night": 550_000,   "area": "Quận 3",      "rating": 4.4},
        {"name": "The Common Room",       "stars": 2, "price_per_night": 180_000,   "area": "Quận 1",      "rating": 4.6},
    ],
}


@tool
def search_flights(origin: str, destination: str) -> str:
    """
    Tìm kiếm các chuyến bay giữa hai thành phố.
    Tham số:
    - origin: thành phố khởi hành (VD: 'Hà Nội', 'Hồ Chí Minh')
```

```
    - destination: thành phố đến (VD: 'Đà Nẵng', 'Phú Quốc')
    Trả về danh sách chuyến bay với hãng, giờ bay, giá vé.
    Nếu không tìm thấy tuyến bay, trả về thông báo không có chuyến.
    """
    # TODO: Sinh viên tự triển khai
    # - Tra cứu FLIGHTS_DB với key (origin, destination)
    # - Nếu tìm thấy → format danh sách chuyến bay dễ đọc, bao gồm giá
tiền
    # - Nếu không tìm thấy → thử tra ngược (destination, origin) xem có
không,
    #   nếu cũng không có → "Không tìm thấy chuyến bay từ X đến Y."
    # - Gợi ý: format giá tiền có dấu chấm phân cách (1.450.000đ)
    pass


@tool
def search_hotels(city: str, max_price_per_night: int = 99999999) -> str:
    """
    Tìm kiếm khách sạn tại một thành phố, có thể lọc theo giá tối đa mỗi
đêm.
    Tham số:
    - city: tên thành phố (VD: 'Đà Nẵng', 'Phú Quốc', 'Hồ Chí Minh')
    - max_price_per_night: giá tối đa mỗi đêm (VNĐ), mặc định không giới
hạn
    Trả về danh sách khách sạn phù hợp với tên, số sao, giá, khu vực,
rating.
    """
    # TODO: Sinh viên tự triển khai
    # - Tra cứu HOTELS_DB[city]
    # - Lọc theo max_price_per_night
    # - Sắp xếp theo rating giảm dần
    # - Format đẹp. Nếu không có kết quả → "Không tìm thấy khách sạn tại
X
    #   với giá dưới Y/đêm. Hãy thử tăng ngân sách."
    pass


@tool
def calculate_budget(total_budget: int, expenses: str) -> str:
    """
```

```
    Tính toán ngân sách còn lại sau khi trừ các khoản chi phí.
    Tham số:
    - total_budget: tổng ngân sách ban đầu (VNĐ)
    - expenses: chuỗi mô tả các khoản chi, mỗi khoản cách nhau bởi dấu
phẩy,
      định dạng 'tên_khoản:số_tiền' (VD:
'vé_máy_bay:890000,khách_sạn:650000')
    Trả về bảng chi tiết các khoản chi và số tiền còn lại.
    Nếu vượt ngân sách, cảnh báo rõ ràng số tiền thiếu.
    """
    # TODO: Sinh viên tự triển khai
    # - Parse chuỗi expenses thành dict {tên: số_tiền}
    # - Tính tổng chi phí
    # - Tính số tiền còn lại = total_budget - tổng chi phí
    # - Format bảng chi tiết:
    #     Bảng chi phí:
    #     - Vé máy bay: 890.000đ
    #     - Khách sạn: 650.000đ
    #     ---
    #     Tổng chi: 1.540.000đ
    #     Ngân sách: 5.000.000đ
    #     Còn lại: 3.460.000đ
    # - Nếu âm → "Vượt ngân sách X đồng! Cần điều chỉnh."
    # - Xử lý lỗi: nếu expenses format sai → trả về thông báo lỗi rõ ràng
    pass

--- KẾT THÚC CODE tools.py ---
```

Chú ý:

- search_flights: phải xử lý tuple key, thử tra ngược chiều
- search_hotels: phải lọc + sắp xếp, không chỉ lookup
- calculate_budget: phải parse chuỗi, xử lý format lỗi, tính toán thực sự
- 3 tools có MỐI LIÊN HỆ: kết quả flights → input cho budget → quyết định hotels

==========

PHẦN 3: TRIỂN KHAI LANGGRAPH (60 Phút)

==========

Mục tiêu: Tạo Vòng lặp Agent — Agent tự quyết định gọi tool nào, bao nhiêu lần.

Nhiệm vụ: Tạo file agent.py

```
--- BẮT ĐẦU CODE agent.py ---

from typing import Annotated
from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.graph.message import add_messages
from langgraph.prebuilt import ToolNode, tools_condition
from langchain_openai import ChatOpenAI
from langchain_core.messages import SystemMessage
from tools import search_flights, search_hotels, calculate_budget
from dotenv import load_dotenv

load_dotenv()

# 1. Đọc System Prompt
with open("system_prompt.txt", "r", encoding="utf-8") as f:
    SYSTEM_PROMPT = f.read()

# 2. Khai báo State
class AgentState(TypedDict):
    messages: Annotated[list, add_messages]

# 3. Khởi tạo LLM và Tools
tools_list = [search_flights, search_hotels, calculate_budget]
llm = ChatOpenAI(model="gpt-4o-mini")
llm_with_tools = llm.bind_tools(tools_list)

# 4. Agent Node
def agent_node(state: AgentState):
    messages = state["messages"]
    if not isinstance(messages[0], SystemMessage):
        messages = [SystemMessage(content=SYSTEM_PROMPT)] + messages

    response = llm_with_tools.invoke(messages)
```

```python
    # === LOGGING ===
    if response.tool_calls:
        for tc in response.tool_calls:
            print(f"Gọi tool: {tc['name']}({tc['args']})")
    else:
        print(f"Trả lời trực tiếp")

    return {"messages": [response]}

# 5. Xây dựng Graph
builder = StateGraph(AgentState)
builder.add_node("agent", agent_node)

tool_node = ToolNode(tools_list)
builder.add_node("tools", tool_node)

# TODO: Sinh viên khai báo edges
# builder.add_edge(START, ...)
# builder.add_conditional_edges("agent", tools_condition)
# builder.add_edge("tools", ...)

graph = builder.compile()

# 6. Chat loop
if __name__ == "__main__":
    print("=" * 60)
    print("TravelBuddy — Trợ lý Du lịch Thông minh")
    print("   Gõ 'quit' để thoát")
    print("=" * 60)

    while True:
        user_input = input("\nBạn: ").strip()
        if user_input.lower() in ("quit", "exit", "q"):
            break

        print("\nTravelBuddy đang suy nghĩ...")
        result = graph.invoke({"messages": [("human", user_input)]})
        final = result["messages"][-1]
        print(f"\nTravelBuddy: {final.content}")
```

```
--- KẾT THÚC CODE agent.py ---
```

==========

PHẦN 4: TEST CASES & CHẨN ĐOÁN (45 Phút)

==========

Sinh viên chạy agent và nhập 5 kịch bản sau, chụp/copy kết quả:

```
Test 1 — Direct Answer (Không cần tool)
  User: "Xin chào! Tôi đang muốn đi du lịch nhưng chưa biết đi đâu."
  Kỳ vọng: Agent chào hỏi, hỏi thêm về sở thích/ngân sách/thời gian.
  Không gọi tool nào.

Test 2 — Single Tool Call
  User: "Tìm giúp tôi chuyến bay từ Hà Nội đi Đà Nẵng"
  Kỳ vọng: Gọi search_flights("Hà Nội", "Đà Nẵng"), liệt kê 4 chuyến bay.

Test 3 — Multi-Step Tool Chaining
  User: "Tôi ở Hà Nội, muốn đi Phú Quốc 2 đêm, budget 5 triệu. Tư vấn
giúp!"
  Kỳ vọng: Agent phải tự chuỗi nhiều bước:
  1. search_flights("Hà Nội", "Phú Quốc") → tìm vé rẻ nhất (1.100.000đ)
  2. search_hotels("Phú Quốc", max_price phù hợp) → gợi ý trong tầm giá
  3. calculate_budget(5000000, "vé_bay:1100000,khách_sạn:...") → tính còn
lại
  Rồi tổng hợp thành gợi ý hoàn chỉnh với bảng chi phí.

Test 4 — Missing Info / Clarification
  User: "Tôi muốn đặt khách sạn"
  Kỳ vọng: Agent hỏi lại: thành phố nào? bao nhiêu đêm? ngân sách bao
nhiêu?
  Không gọi tool vội.

Test 5 — Guardrail / Refusal
  User: "Giải giúp tôi bài tập lập trình Python về linked list"
  Kỳ vọng: Từ chối lịch sự, nói rằng chỉ hỗ trợ về du lịch.
```

==========

PHẦN 5: DELIVERABLES & ĐÁNH GIÁ (15 Phút)

==========

Nộp bài: Nén thành MSSV_Lab4.zip gồm:

- system_prompt.txt
- tools.py
- agent.py
- test_results.md — Copy/paste console log của ít nhất 5 test cases

Rubric:

Tiêu chí	Điểm
Setup LangGraph đúng (Nodes, Edges, Graph chạy được)	25%
Tool implementations đúng logic + xử lý lỗi (try/except)	25%
System Prompt kiên cố (vượt Test 4 + Test 5)	20%
Agent thực hiện multi-step tool chaining thành công (Test 3)	20%
Code sạch, có type hints, logging rõ ràng	10%